

# Least Squares Regression (Part 2)

## Curve fitting with a high order polynomial

In Part 1, we have learnt about linear regression using the 1<sup>st</sup> order polynomial:

$$\hat{\mathbf{z}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}, \quad \hat{\mathbf{y}} = \mathbf{A} \hat{\mathbf{z}} = \mathbf{A} (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

Now, we can expand this least square regression method to a higher order polynomial.

$n^{\text{th}}$  order polynomial can be represented with  $m$  data points as

$$f(x_k) = a_0 + a_1 x_k + a_2 x_k^2 + \cdots + a_n x_k^n, \quad k=1 \text{ to } m$$

The matrix  $\mathbf{A}$  and vector  $\mathbf{z}$  for  $n^{\text{th}}$  order polynomial is then expressed as

$\mathbf{y} = \mathbf{A} \mathbf{z}$ , where

$$\mathbf{A} = \begin{bmatrix} 1 & x_1 & \cdots & x_1^{n-1} & x_1^n \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_m & \cdots & x_m^{n-1} & x_m^n \end{bmatrix}, \quad \text{and} \quad \mathbf{z} = \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix}$$

where Matrix  $\mathbf{A}$  is the Vandermonde matrix.

However, as the order  $n$  increases, *it becomes more ill-conditioned*, where columns of the matrix  $\mathbf{A}$  can become almost linearly dependent.

## Procedure

Aim: Find the optimal parameter  $\mathbf{z}$  that gives the least residual error:

$$\hat{\mathbf{z}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}, \quad \hat{\mathbf{y}} = \mathbf{A} \hat{\mathbf{z}} = \mathbf{A} (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

For  $n^{\text{th}}$  order polynomial, express the parameter vector  $\mathbf{z}$  and matrix  $\mathbf{A}$ , where

$$\mathbf{A} = \begin{bmatrix} 1 & x_1 \cdots & x_1^{n-1} & x_1^n \\ \vdots & \ddots & \vdots & \\ 1 & x_m \cdots & x_m^{n-1} & x_m^n \end{bmatrix}, \quad \mathbf{z} = \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

Matrix ( $\mathbf{A}^T$ )

Vector ( $\mathbf{A}^T \mathbf{y}$ )

$$\mathbf{A}^T = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 \\ \vdots & & & & \vdots \\ x_1^{n-1} & x_2^{n-1} & & x_{m-1}^{n-1} & x_m^{n-1} \\ x_1^n & x_2^n & \cdots & x_{m-1}^n & x_m^n \end{bmatrix} \quad \mathbf{A}^T \mathbf{y} = \begin{bmatrix} \sum_{k=1}^m y_k \\ \sum_{k=1}^m y_k x_k^1 \\ \vdots \\ \sum_{k=1}^m y_k x_k^{n-1} \\ \sum_{k=1}^m y_k x_k^n \end{bmatrix}$$

Matrix ( $\mathbf{A}^T \mathbf{A}$ )

Since  $\mathbf{A}^T \mathbf{A}$  is symmetric, we need to find only the upper diagonal terms.

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} 1 & 1 & \cdots & 1 & 1 \\ \vdots & & & & \vdots \\ x_1^{n-1} & x_2^{n-1} & & x_{m-1}^{n-1} & x_m^{n-1} \\ x_1^n & x_2^n & \cdots & x_{m-1}^n & x_m^n \end{bmatrix} \begin{bmatrix} 1 & x_1 \cdots & x_1^{n-1} & x_1^n \\ \vdots & \ddots & \vdots & \\ 1 & x_m \cdots & x_m^{n-1} & x_m^n \end{bmatrix}$$

$$\mathbf{A}^T \mathbf{A} = \begin{bmatrix} m & \sum_{k=1}^m x_k^1 & \cdots & \sum_{k=1}^m x_k^{n-1} & \sum_{k=1}^m x_k^n \\ \sum_{k=1}^m x_k^1 & \sum_{k=1}^m x_k^2 & \cdots & \sum_{k=1}^m x_k^n & \sum_{k=1}^m x_k^{n+1} \\ \vdots & \vdots & & \vdots & \vdots \\ \sum_{k=1}^m x_k^{n-1} & \sum_{k=1}^m x_k^n & \cdots & \sum_{k=1}^m x_k^{2n-2} & \sum_{k=1}^m x_k^{2n-1} \\ \sum_{k=1}^m x_k^n & \sum_{k=1}^m x_k^{n+1} & \cdots & \sum_{k=1}^m x_k^{2n-1} & \sum_{k=1}^m x_k^{2n} \end{bmatrix}$$

Then, apply the inverse of  $\mathbf{A}^T \mathbf{A}$  using previous linear equation methods to obtain

$$\hat{\mathbf{z}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}, \quad \hat{\mathbf{y}} = \mathbf{A} \hat{\mathbf{z}}$$

Q. Write down a pseudocode for the linear least square method for nth order polynomial

```
// [ z, E] = polyfit(e , x, y, n)
// returns the parameter vector: z=[a0, a1 a2,...an]
//  $f(x_k) = a_0 + a_1 x_k + a_2 x_k^2 + \dots + a_n x_k^n$ 
// E: total residual error E
// n: order of polynomial.  $1 \leq n < m$ 
// x, y: 1-D array of data points (m sets of points)

for (int i=0 ; i < 2n; i++) {
    for (int k=0; k < m; k++) {
        Sx[i][0] += pow(x[k], i)
    }
}
```

## MATLAB example code: nth order polynomial least square

*This is NOT an efficient code. Make it more efficient.*

```
% 생략
% 생략

% A: (m) by (n+1) // A'A: (n+1) by (n+1)
% number of data sets m
m = length(x);
% polynomial order (a4)x^4+(a3)x^3+...+(a1)x+a0.
n = 4;

% Calculate Sum of x^1 to x^(2n)
% It is Diagonal terms of A'A (except last element)
for i = 1:2*n
    xsum(i) = sum(x.^(i));
end

% S*z=b, S=A'A, b=(A'y)
% First row of S
S(1,1) = m;
for j= 2:n+1
    S(1,j)=xsum(j-1);
end

% First element of b
b(1,1)= sum(y);

% From 2nd row of S and b
for i = 2:n+1
    for j=1:n+1
        S(i,j)= xsum(i+j-2);
    end
    b(i,1) = sum(x.^(i-1).*y);
end
```

```
% Find the inverse of S or use Gauss Elim.  
z=(S\b) '
```

```
% z=[a0, a1 a2 a3 a4....]  
% Pcoef=[....a4 a3 a2 a1 a0]  
for i = 1:n+1  
    Pcoef(i) = z(n+2-i);  
end
```

## General Case of Least Square Regression

We can expand the linear least-square regression to a general curve-fit using a family of linear functions

The curve fit function  $f(x)$  can be expressed in terms of linear functions  $g(x)$ . Here, the target parameters to be solved are  $c_j$ .

Thus, the basis functions  $g(x)$  can be either linear and non-linear as

$$f(x) = \hat{y} = \sum_{j=0}^n c_j g_j(x)$$

where  $c_j$  are coefficients of the best fit curve and  $g(x)$  are the basis functions

- **Example:**

Curve fit of high polynomial:  $f(x_k) = a_0 + a_1 x_k + a_2 x_k^2 + \dots + a_n x_k^n$

|                    |             |
|--------------------|-------------|
| $g_0(x_k) = 1$     | $c_0 = a_0$ |
| $g_1(x_k) = x_k$   | $c_1 = a_1$ |
| $g_2(x_k) = x_k^2$ | $c_2 = a_2$ |
| $\vdots$           | $\vdots$    |
| $g_n(x_k) = x_k^n$ | $c_n = a_n$ |

Curve fit of non-linear functions  $f(x_k) = a_0 \ln x + a_1 \cos x + a_2 e^x + \dots$

|                     |             |
|---------------------|-------------|
| $g_0(x_k) = \ln x$  | $c_1 = a_1$ |
| $g_1(x_k) = \cos x$ | $c_2 = a_2$ |
| $g_2(x_k) = e^x$    | $\vdots$    |
| $\vdots$            |             |

Let there are  $m$  sets of data points  $(x_k, y_k)$  where  $k= 1$  to  $m$ .

$$E = \sum_{k=1}^m (y_k - f(x_k))^2 = \sum_{k=1}^m \left( y_k - \sum_{j=0}^n c_j g_j(x_k) \right)^2$$

To find the minimum point of  $E$  with respect to  $c_i$

$$\frac{\partial E}{\partial c_i} = -2 \sum_{k=1}^m \left( y_k - \sum_{j=0}^n c_j g_j(x_k) \right) g_i(x_k) = 0, \quad i = 0 \text{ to } n$$

$$\begin{aligned} \sum_{k=1}^m y_k g_i(x_k) &= \sum_{k=1}^m \left( \sum_{j=0}^n c_j g_j(x_k) \right) g_i(x_k) \\ &= \sum_{k=1}^m g_i(x_k) (c_0 g_0(x_k) + c_1 g_1(x_k) + \cdots + c_n g_n(x_k)) \end{aligned}$$

This can be expressed in matrix form of

$$\begin{bmatrix} \sum_{k=1}^m g_0^2(x_k) & \sum_{k=1}^m g_0(x_k)g_1(x_k) & \cdots & \sum_{k=1}^m g_0(x_k)g_n(x_k) \\ \sum_{k=1}^m g_1(x_k)g_0(x_k) & \sum_{k=1}^m g_1^2(x_k) & & \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m g_n(x_k)g_0(x_k) & \sum_{k=1}^m g_n(x_k)g_1(x_k) & \cdots & \sum_{k=1}^m g_n^2(x_k) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} \sum_{k=1}^m y_k g_0(x_k) \\ \sum_{k=1}^m y_k g_1(x_k) \\ \vdots \\ \sum_{k=1}^m y_k g_n(x_k) \end{bmatrix}$$

Also, using  $\mathbf{A} = \begin{bmatrix} g_0(x_1) & g_1(x_1) \cdots & g_{n-1}(x_1) & g_n(x_1) \\ \vdots & \ddots & \vdots & \\ g_0(x_m) & g_1(x_m) \cdots & g_{n-1}(x_m) & g_n(x_m) \end{bmatrix}$ , and  $\mathbf{z} = \begin{bmatrix} c_0 \\ \vdots \\ c_n \end{bmatrix}$

the normal equation becomes

$$(\mathbf{A}^T \mathbf{A}) \mathbf{z} = \mathbf{A}^T \mathbf{y}$$

$$\rightarrow \mathbf{z} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

Thus, outputs on the fitted curve are then

$$\hat{\mathbf{y}} = \mathbf{A} \mathbf{z},$$

- Size of matrix  $\mathbf{A}$ :  $(m) \times (n+1)$
- Size of matrix  $\mathbf{A}^T \mathbf{A}$ :  $(n+1) \times (n+1)$ , for  $n^{\text{th}}$  order functions
- Since  $(\mathbf{A}^T \mathbf{A})$  is symmetric, need to know only the upper diagonal terms.
- The basis functions  $g_i(x)$  must be independent to one another.

**Q.** If the basis  $g_i(x)$  are all orthogonal basis then how would the matrix  $(\mathbf{A}^T \mathbf{A})$  change?

What is the expression for  $c_i$ ?

Hint: if  $g_i(x)$  and  $g_j(x)$  are orthogonal, then  $\sum_{k=1}^m g_i(x_k) g_j(x_k) = 0$

$$\begin{bmatrix} \sum_{k=1}^m g_0^2(x_k) & 0 & \dots & 0 \\ 0 & \sum_{k=1}^m g_1^2(x_k) & & 0 \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \dots & \sum_{k=1}^m g_n^2(x_k) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} \sum_{k=1}^m y_k g_0(x_k) \\ \sum_{k=1}^m y_k g_1(x_k) \\ \vdots \\ \sum_{k=1}^m y_k g_n(x_k) \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} c_0 \sum g_0^2 \\ c_1 \sum g_1^2 \\ \vdots \\ c_n \sum g_n^2 \end{bmatrix} = \begin{bmatrix} \sum y_k g_0 \\ \sum y_k g_1 \\ \vdots \\ \sum y_k g_n \end{bmatrix} \quad \therefore c_i = \frac{\sum y_k g_i}{\sum g_i^2}$$



# Curve Fitting with other non-linear functions

Need to make the best fit function parameters to be represented in the linear form

Commonly used non-linear functions:

| Nonlinear equation   | Linear form                                           | Relationship to $Y = a_1X + a_0$                                             | Values for linear least-squares regression | Plot where data points appear to fit a straight line                                                           |
|----------------------|-------------------------------------------------------|------------------------------------------------------------------------------|--------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| $y = bx^m$           | $\ln(y) = m\ln(x) + \ln(b)$                           | $Y = \ln(y), X = \ln(x)$<br>$a_1 = m, a_0 = \ln(b)$                          | $\ln(x_i)$ and $\ln(y_i)$                  | $y$ vs. $x$ plot on logarithmic $y$ and $x$ axes.<br>$\ln(y)$ vs. $\ln(x)$ plot on linear $x$ and $y$ axes.    |
| $y = be^{mx}$        | $\ln(y) = mx + \ln(b)$                                | $Y = \ln(y), X = x$<br>$a_1 = m, a_0 = \ln(b)$                               | $x_i$ and $\ln(y_i)$                       | $y$ vs. $x$ plot on logarithmic $y$ and linear $x$ axes.<br>$\ln(y)$ vs. $x$ plot on linear $x$ and $y$ axes.  |
| $y = b10^{mx}$       | $\log(y) = mx + \log(b)$                              | $Y = \log(y), X = x$<br>$a_1 = m, a_0 = \log(b)$                             | $x_i$ and $\log(y_i)$                      | $y$ vs. $x$ plot on logarithmic $y$ and linear $x$ axes.<br>$\log(y)$ vs. $x$ plot on linear $x$ and $y$ axes. |
| $y = \frac{1}{mx+b}$ | $\frac{1}{y} = mx + b$                                | $Y = \frac{1}{y}, X = x$<br>$a_1 = m, a_0 = b$                               | $x_i$ and $1/y_i$                          | $1/y$ vs. $x$ plot on linear $x$ and $y$ axes.                                                                 |
| $y = \frac{mx}{b+x}$ | $\frac{1}{y} = \frac{b}{m} \frac{1}{x} + \frac{1}{m}$ | $Y = \frac{1}{y}, X = \frac{1}{x}$<br>$a_1 = \frac{b}{m}, a_0 = \frac{1}{m}$ | $1/x_i$ and $1/y_i$                        | $1/y$ vs. $1/x$ plot on linear $x$ and $y$ axes.                                                               |

Example: Curve-fit in the exponential form of  $y = be^{mx}$

- Rearrange to the linear form of  $\ln(y) = mx + \ln(b)$

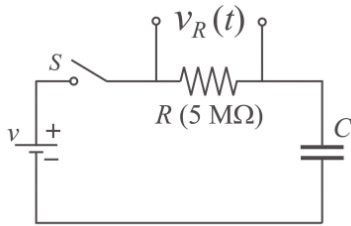
$$\rightarrow f(x) = \hat{y} = a_1x + a_0 \quad \text{where } f(x) = \ln(y), \quad a_1 = m, \quad a_0 = \ln(b)$$

- Solve for  $a_1, a_0$  using linear least square. Then covert back to  $m, b$

Exercise: Curve-fit the voltage across the resistor in 1<sup>st</sup> order RC circuit.

How can you find the best parameters of capacitance C and initial voltage  $V_0$ ?

Express as  $\hat{\mathbf{c}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$  and  $\hat{\mathbf{y}} = \mathbf{A} \hat{\mathbf{c}}$



$$V_R(t) = V_0 e^{-t/RC}$$

$$V_R = V_0 e^{-t/RC} = V_0 e^{-\frac{t}{\tau}}$$

$$\ln(V_R) = \ln(V_0) + \left(-\frac{1}{\tau} \cdot t\right)$$

$$f(t) = a_0 + a_1 t \quad \Rightarrow \quad \tau = RC = -\frac{1}{a_1} \Rightarrow C = \frac{1}{R a_1}$$

$$f(t) = \ln V_R$$

$$V_0 = e^{a_0}$$

$$a_0 = \ln V_0$$

$$a_1 = -\frac{1}{\tau} = -\frac{1}{RC}$$

$$\therefore V_R = V_0 e^{-\frac{t}{\tau}}$$

# Summary

Aim: minimize  $\|\mathbf{y} - \hat{\mathbf{y}}\|^2 (= \|\mathbf{y} - \mathbf{A}\mathbf{z}\|^2)$

---

Curve-fitting with a linear least square regression  $f(x) = \hat{y} = a_0 + a_1x$

$$a_0 = \frac{S_{xx}S_y - S_xS_{xy}}{mS_{xx} - S_x^2}, \quad a_1 = \frac{mS_{xy} - S_xS_y}{mS_{xx} - S_x^2}$$

This is equivalent to

$$\hat{\mathbf{z}} = \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} m & \sum_{k=1}^m x_k \\ \sum_{k=1}^m x_k & \sum_{k=1}^m x_k^2 \end{bmatrix}^{-1} \begin{bmatrix} \sum_{k=1}^m y_k \\ \sum_{k=1}^m x_k y_k \end{bmatrix} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}, \quad \text{where } \mathbf{A} = \begin{bmatrix} 1 & x_1 \\ \vdots & \vdots \\ 1 & x_m \end{bmatrix}$$


---

---

Curve-fitting with a nth order polynomial  $f(x_k) = a_0 + a_1x_k + a_2x_k^2 + \cdots + a_nx_k^n$

$$\hat{\mathbf{z}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}, \quad \hat{\mathbf{y}} = \mathbf{A}\hat{\mathbf{z}},$$

$$\mathbf{z} = \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & x_1 & \cdots & x_1^{n-1} & x_1^n \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_m & \cdots & x_m^{n-1} & x_m^n \end{bmatrix}, \quad \mathbf{A}^T \mathbf{A} = \begin{bmatrix} m & \sum_{k=1}^m x_k & \cdots & \sum_{k=1}^m x_k^{n-1} & \sum_{k=1}^m x_k^n \\ \sum_{k=1}^m x_k & \sum_{k=1}^m x_k^2 & \cdots & \sum_{k=1}^m x_k^n & \sum_{k=1}^m x_k^{n+1} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \sum_{k=1}^m x_k^{n-1} & \sum_{k=1}^m x_k^n & \cdots & \sum_{k=1}^m x_k^{2n-2} & \sum_{k=1}^m x_k^{2n-1} \\ \sum_{k=1}^m x_k^n & \sum_{k=1}^m x_k^{n+1} & \cdots & \sum_{k=1}^m x_k^{2n-1} & \sum_{k=1}^m x_k^{2n} \end{bmatrix}, \quad \mathbf{A}^T \mathbf{y} = \begin{bmatrix} \sum_{k=1}^m y_k \\ \sum_{k=1}^m y_k x_k \\ \vdots \\ \sum_{k=1}^m y_k x_k^{n-1} \\ \sum_{k=1}^m y_k x_k^n \end{bmatrix}$$


---

---

Curve-fitting with basis functions  $g(x)$   $f(x) = \hat{y} = \sum_{j=0}^n c_j g_j(x)$

$$\mathbf{z} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{y}$$

$$\begin{bmatrix} \sum_{k=1}^m g_0^2(x_k) & \sum_{k=1}^m g_0(x_k)g_1(x_k) & \cdots & \sum_{k=1}^m g_0(x_k)g_n(x_k) \\ \sum_{k=1}^m g_1(x_k)g_0(x_k) & \sum_{k=1}^m g_1^2(x_k) & & \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{k=1}^m g_n(x_k)g_0(x_k) & \sum_{k=1}^m g_n(x_k)g_1(x_k) & \cdots & \sum_{k=1}^m g_n^2(x_k) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} \sum_{k=1}^m y_k g_0(x_k) \\ \sum_{k=1}^m y_k g_1(x_k) \\ \vdots \\ \sum_{k=1}^m y_k g_n(x_k) \end{bmatrix}$$

where

$$\mathbf{A} = \begin{bmatrix} g_0(x_1) & g_1(x_1) \cdots & g_{n-1}(x_1) & g_n(x_1) \\ \vdots & \ddots & \vdots & \\ g_0(x_m) & g_1(x_m) \cdots & g_{n-1}(x_m) & g_n(x_m) \end{bmatrix}, \text{ and } \mathbf{z} = \begin{bmatrix} c_0 \\ \vdots \\ c_n \end{bmatrix}$$


---